

MiTienda Pro

Sistema de Ventas + PWA + Sincronización GitHub

Guía completa del desarrollador

Edrian Cruz Down · 2026

1. Arquitectura y estructura de carpetas

MiTienda Pro es una **aplicación 100% cliente** (HTML + CSS + JS) que se publica en GitHub Pages y funciona como PWA instalable. **No hay servidor backend**: el propio repositorio de GitHub actúa como base de datos pública (lectura) y privada (escritura con token).

Estructura del repositorio

```
mi-repo/
├── app/                                ← Código del PWA publicado en GitHub Pages
│   ├── index.html                     ← HTML principal (limpio, sin JS/CSS inline)
│   ├── manifest.json                  ← Manifiesto PWA (instalable)
│   ├── service-worker.js              ← Cache offline (v1.4.0)
│   ├── css/styles.css                 ← Estilos extraídos
│   └── js/
│       ├── main.js                    ← Lógica del sitio + admin
│       ├── guard.js                   ← Anti-DevTools y branding
│       └── lib/                        ← Capa PWA + GitHub (módulos ES6)
│           ├── bootstrap.js           ← Punto de arranque, registra SW, expone window.MTP
│           ├── crypto.js               ← AES-GCM + PBKDF2 (encripta el token)
│           ├── indexeddb.js            ← Storage offline (config, cache, queue, logs, backups)
│           ├── github-api.js           ← Cliente REST de GitHub (PAT)
│           ├── reader.js               ← Lectura de /public con cache-first
│           ├── writer.js               ← Escritura JSON e imágenes (encola si offline)
│           ├── sync.js                 ← Vacía la cola cada 30s
│           ├── gh-sync.js              ← Sincroniza el estado S ↔ public/data/state.json
│           ├── repo-init.js            ← Crea /app /public /private + state.json inicial
│           ├── image-opt.js            ← Compresión + WebP + miniaturas (Canvas)
│           ├── backup.js               ← Snapshots a /private/backups
│           ├── admin-config.js         ← UI dentro de Admin → Avanzado para conectar GitHub
│           ├── admin-fix.js            ← Arregla pequeños bugs del admin original
│           └── mobile-menu.js          ← Hamburguesa móvil (≤900px)
├── public/                             ← Datos públicos servidos por GitHub Pages
│   ├── data/
│   │   ├── state.json                 ← Estado sincronizado entre dispositivos
│   │   └── img/                       ← Avatar, icono, fondo de contacto, etc.
└── private/                            ← Backups (no se sirve por Pages)
    └── backups/
```

¿Qué es la “raíz” del repositorio?

La **raíz** es la carpeta más alta: la que contiene **app/**, **public/** y **private/**. Cuando publicas en GitHub Pages, la URL final es:

```
https://<usuario>.github.io/<repo>/app/
```

Por eso todo lo que está en **/app** se ve por web, y todo lo que está en **/public** se sirve también para que los demás dispositivos lo descarguen (state.json, imágenes, etc.).

2. Sincronización entre dispositivos

Idea general

El estado completo del admin (planes, contactos, links, avatar, vendedores, tiendas, testimonios, etc.) vive en una variable global **S**. Esa variable se serializa a JSON y se publica en **public/data/state.json** dentro del repositorio. Cualquier dispositivo que abra la URL del sistema descarga ese JSON y lo aplica encima de los valores por defecto.

Flujo PUSH (admin → GitHub)

- El admin guarda algo (avatar, plan, contacto...). **saveS()** persiste local en localStorage.
- **gh-sync.js** tiene un wrap sobre saveS que encola un push con debounce de 5s.
- **pushNow()** lee S, lo limpia (sin contraseñas), añade `__ts` (timestamp) y llama a `Writer.writeJSON`.
- `Writer` encripta el commit con el token PAT (encriptado con AES-GCM + tu passphrase) y hace PUT a la API de GitHub.
- Si el dispositivo está offline, queda en **queue** de IndexedDB y se reintenta cada 30s y al volver online.

Flujo PULL (otro dispositivo → ve los cambios)

- Al cargar la app, **gh-sync.install()** detecta el repo automáticamente desde la URL (`usuario.github.io/repo/app/`).
- Hace fetch de `../public/data/state.json` (relativo) y como fallback **raw.githubusercontent.com**.
- Mergea contra la S local: si el remoto tiene `__ts` mayor, prevalece; si no, solo rellena campos vacíos.
- Llama a **renderAll()** para repintar el sitio sin recargar.
- Repite cada 60 segundos y al evento **online**.

Imágenes (avatar, icono, fondo)

Antes las imágenes se guardaban como base64 en localStorage y nunca llegaban al otro dispositivo. Ahora cuando subes una imagen desde el admin:

- Se comprime con **image-opt.js** a WebP (≤ 320 px en avatar, ≤ 128 en icono...).
- Se sube como archivo real a **public/img/<tipo>-<timestamp>.webp**.
- En S queda guardada solo la URL relativa `../public/img/...`
- Al sincronizarse el state.json, los demás dispositivos cargan la imagen desde GitHub Pages.

3. Configuración inicial (paso a paso)

3.1 Crear el repositorio en GitHub

- Crea un repo **público** (necesario para que GitHub Pages sirva /public sin auth).
- Sube el contenido del ZIP. La carpeta **app/** debe estar en la raíz, junto a **public/** y **private/**.
- Settings → Pages → Source: **Deploy from a branch**, Branch **main**, Folder **/ (root)**.
- Espera 1-2 min y abre <https://<usuario>.github.io/<repo>/app/>.

3.2 Generar el Personal Access Token (PAT)

- github.com → Settings → Developer settings → Personal access tokens → **Tokens (classic)**.
- Generate new token (classic). Scope: marca solo **repo** (acceso completo al repo).
- Cópialo (empieza por ghp_). **Solo se ve una vez.**

3.3 Conectar la app con tu repo

- Abre la URL del sistema → botón **Admin** → contraseña (por defecto admin2025).
- Ve a la pestaña **Avanzado** → tarjeta **■ GitHub / Sincronización PWA**.
- Rellena **Owner**, **Repositorio**, **Rama** (main), pega el **Token** y elige una **Passphrase** fuerte.
- **Guardar config + token** → publica automáticamente el primer state.json.
- **Crear /app /public /private** (solo la primera vez si las carpetas no existen).
- **Probar conexión** debe mostrar el nombre del repo en verde.

3.4 Verificación

- Edita un plan o sube un avatar → en 5-10s aparece commit nuevo en GitHub: *state: sync 2026-...*
- Abre la URL en otro dispositivo (móvil): el cambio se ve sin tocar nada.
- Si no aparece: Admin → Avanzado → log verde inferior. Mostrará el motivo exacto (token, passphrase, etc.).

4. Modo offline

El service-worker (v1.4.0) cachea el shell completo de la app. Estrategias:

Tipo de recurso	Estrategia	Comportamiento
HTML (navegación)	Network-first 6s	Si falla, sirve index.html cacheado
CSS/JS/lib propios	Stale-while-revalidate	Sirve cache, refresca en 2º plano
Fuentes Google	Cache-first	Una vez bajadas no se vuelven a pedir
api.github.com	Network-first 5s	No se sirve cache si caduca
raw.githubusercontent.com	Stale-while-revalidate	Permite ver state.json offline
Imágenes (.webp/.png)	Cache-first	Avatar/icono offline siempre

Cómo se prueba

- Abre la app en Windows y déjala cargar (la primera vez baja todo).
- Activa **modo avión** y refresca: debe seguir funcionando completa.
- En Android: instálala (Chrome → menú → **Añadir a pantalla de inicio**) y úsala sin datos.
- En el panel admin tienes **■ Limpiar caché PWA** si necesitas forzar recarga.

5. Generar la APK Android

Opción A: PWABuilder (recomendado, sin instalar nada)

- Publica el sitio en GitHub Pages.
- Entra en <https://www.pwabuilder.com> y pega tu URL.
- Pulsa **Package for stores** → **Android** → **Generate**.
- Descarga el ZIP, dentro está el **.apk** firmada.
- Súbela a Telegram/WhatsApp para instalarla en clientes.

Opción B: Bubblewrap CLI (más control)

```
npm i -g @bubblewrap/cli
bubblewrap init --manifest=https://<user>.github.io/<repo>/app/manifest.json
bubblewrap build
# Genera app-release-signed.apk firmada con tu keystore.
```

Requisitos del manifest.json

- **start_url** y **scope** apuntando a `./`.
- Iconos 192x192 y 512x512 (incluidos en `/app/icons/` — añádelos si no existen).
- **display: standalone** para abrir sin barra del navegador.

6. Operación diaria del admin

Cambiar contraseña del admin

Admin → Configuración → Seguridad → Cambiar contraseña. Se guarda encriptada en local y se sincroniza.

Copia de seguridad

- Admin → Avanzado → ■ **Backup ahora**: sube un JSON timestamped a **private/backups/**.
- Admin → Configuración → **Exportar JSON**: descarga el estado completo a tu PC.
- Admin → Configuración → **Importar JSON**: restaura desde un backup local.

Forzar sincronización

Admin → Avanzado → ■ **Sincronizar ahora**. Vacía toda la cola pendiente al instante.

Resetear todo el sistema

Admin → Configuración → ■ **Resetear sistema**. Limpia localStorage y vuelve a los valores por defecto. Si tienes state.json en GitHub, al recargar lo vuelve a cargar.

7. Solución de problemas (FAQ)

Subo cambios desde Windows pero el móvil no los ve

- Abre Admin → Avanzado → mira el log: ¿dice “push OK”? Si no, falta token o passphrase.
- Comprueba en GitHub: la pestaña Commits debe mostrar “state: sync ...”.
- En el móvil: limpia caché del navegador y recarga. La nueva versión se aplica en menos de 60s.

Error 403 al crear las carpetas

- Tu token PAT no tiene scope **repo**. Genera uno nuevo con scope correcto.
- Si el repo es de una organización, necesitas habilitar **SSO** en el token o usar un PAT con permisos sobre la org.
- Las carpetas solo se crean una vez. Si ya existen, el botón “Crear /app /public /private” no hace nada (es idempotente).

Pantalla blanca offline en Android

- Asegúrate de haber abierto la app al menos una vez con internet (para que el SW cachee).
- Si persiste, Admin → Avanzado → ■ **Limpiar caché PWA** y vuelve a abrir online.
- El SW v1.4.0 ahora incluye fallback explícito a index.html y rutas absolutas; este bug está resuelto.

La app se ve “inicial” aunque haya cambios remotos

- Pasa cuando el repo no se infiere bien (subdominio custom). Solución: en Admin → Avanzado pon Owner/Repo manualmente y guarda — incluso sin token, eso permite el pull.

El avatar/icono no se ve en otro dispositivo

- Versiones antiguas guardaban base64 grande que el sync recortaba. Vuelve a subir la imagen: ahora se guarda como archivo en **public/img/** y se sincroniza correctamente.

8. Ruta de estudio para crear más sistemas

Bases comunes (1 mes)

- HTML5 semántico, CSS3 (Flexbox, Grid, variables, animaciones).
- JavaScript moderno: ES6+, fetch, async/await, módulos, FileReader, IndexedDB.
- Git + GitHub: ramas, PRs, GitHub Pages, GitHub Actions.

E-commerce (2-3 meses)

- React + Vite o Next.js. Routing, contextos, hooks.
- Carrito persistente, checkout, cupones, inventario.
- Pasarelas: Stripe (internacional), Tropipay/Enzona (Cuba), MercadoPago (LatAm).
- Backend: Supabase / Lovable Cloud (auth, DB, storage, edge functions).

Landing page de alto impacto (1-2 semanas)

- Diseño en Figma, paleta limitada (3 colores), tipografía clara.
- Animaciones suaves (Framer Motion / GSAP).
- Optimización Lighthouse 100/100, lazy loading, formularios con validación.

Portafolio profesional (1 mes)

- Sección hero con propuesta de valor clara.
- Casos de estudio con métricas reales (antes/después).
- Blog con MDX, dark mode, contacto integrado.
- Despliegue en Vercel/Netlify con dominio propio.

Chat online en tiempo real (2-3 meses)

- WebSockets (socket.io) o Supabase Realtime.
- Auth, salas, presencia, mensajes con reacciones, archivos.
- Notificaciones push (Web Push API + service worker).
- Encriptación end-to-end (libsodium) si manejas datos sensibles.

Red social tipo Facebook (6-12 meses)

- Modelado de datos: usuarios, posts, comentarios, likes, follows, grupos.
- Feed personalizado (algoritmo simple por engagement + recencia).
- Subida de imágenes/videos optimizada, CDN, moderación.
- Escalabilidad: caching (Redis), búsquedas (Meilisearch/Typesense), colas.

9. Ideas de productos para problemas reales

■ ■ Cuba (alta prioridad — tu mercado)

- **MiTienda Pro** (este sistema) — ya resuelve la falta de plataformas de e-commerce locales.
- **Cola Inteligente** — apps que avisan cuándo llega un producto a tiendas en MLC/CUP.
- **Trueque CUP** — intercambio P2P de bienes/servicios sin pasarela bancaria.
- **OfflineMaps Cuba** — mapas + comercios geolocalizados que funcionan sin datos.
- **Remesas Tracker** — convertidor de divisas (USD/MLC/CUP) con histórico real.
- **RutaCUP** — agendador de transporte (almendrones, gazelles) con valoraciones.

■ ■ Argentina

- **InflaTracker** — registro de precios crowdsourced contra inflación.
- **FacturAR** — facturación electrónica AFIP simplificada para monotributistas.
- **DolarBlue API** — alertas multi-cotización con notificaciones push.

■ ■ España

- **BarrioApp** — comunidad hiperlocal: ofertas vecinos, eventos, grupos de compra.
- **Autonomos.io** — plantillas de IRPF/IVA + integración con Hacienda.
- **AlquilerJusto** — comparador de precios de alquiler por barrio/m².

Cross-mercado

- **Plantillas de tienda online** tipo MiTienda Pro adaptadas por nicho (panadería, peluquería, taller).
- **Sistema de citas + WhatsApp Business** para profesionales independientes.
- **CRM minimalista** en español para PyMEs (sin curva de Salesforce/HubSpot).

10. Referencia rápida: window.MTP

Desde la consola del navegador puedes usar directamente:

```
// Lectura
await MTP.Reader.readJSON("data/state.json");
MTP.Reader.imageUrl("avatar.webp");

// Escritura
await MTP.Writer.writeJSON("data/productos.json", obj, { message: "update" });
await MTP.Writer.writeImage("logo.webp", uint8array);

// Sincronización manual
await MTP.GhSync.pushNow();           // Sube state.json ahora
await MTP.GhSync.pullAndApply();      // Descarga state.json y aplica
await MTP.Sync.flush();               // Vacía cola pendiente

// Backup
await MTP.Backup.upload("manual");

// GitHub directo
await MTP.GitHub.getConfig();
await MTP.GitHub.saveToken(token, passphrase);

// Logs (debug)
await MTP.DB.listLogs(50);
```

Buena práctica: después de cualquier cambio importante, ejecuta `MTP.DB.listLogs(20)` en la consola para ver qué pasó realmente. Los logs incluyen marcas de tiempo y mensajes detallados.